Compositional Verification of Qupit Clifford Circuit Equations

ANONYMOUS AUTHOR(S)

Proving that a circuit normalizer respects an equational theory can duplicate its case analysis. We show how semantic separation avoids this duplication in the Agda formalization accompanying a recent presentation of Clifford circuits in odd prime dimension. The normalizer is a tower of certified coset tables. A semantic observation identifies each representative; completeness one width below identifies its residual. This turns smaller-instance completeness into a method for certifying the next normalization step. A verified semidirect-product construction adds Pauli corrections without extending the symplectic normalizer, and a central extension restores scalar phases. Checked translations recover the original circuit rules, with an interpretation into an independently defined structural Weyl group. A certificate client uses this interpretation to repair projectively valid replacements. A fully checked two-qutrit example reduces twelve gates to six and three entangling gates to one, retaining a nonzero scalar correction. The presentation proofs check in safe Agda without postulates or holes, uniformly in width and odd prime given a primitive-root witness. The contribution is the proof architecture and its algebraic certificate interface; the mathematical rule set and generic composition library are companion work. Correspondence to the specified complex gate matrices remains outside the checked development.

CCS Concepts: • **Theory of computation** → **Type theory**; **Logic and verification**; **Equational logic and rewriting**; **Quantum computation theory**.

Additional Key Words and Phrases: Clifford group, qudits, generators and relations, completeness, normal forms, symplectic group, semidirect product, Agda, formal verification

1 Introduction

A complete equational theory turns semantic equality into a derivation: if two programs have the same meaning, the theory's equations connect them. For a circuit language, this gives a precise contract for a certifying transformation tool. A client can establish equality in a semantic model and obtain a proof using the language's own equations. Completeness does not choose a cheap circuit or an efficient search strategy; it justifies the passage from semantic reasoning back to syntax.

We study the mechanization of this contract for Clifford circuits on *qupits*, quantum systems of odd prime dimension p. The gates are H, S, and CZ, with scalar gates when global phase matters. Bian et al. [Bian et al. 2026] give sixteen rule schemes characterizing these circuits at every width and odd prime. Their argument uses a symplectic normal form, then restores Pauli and scalar corrections. Checking the primitive equations establishes soundness. Completeness also requires a normalization procedure, a derivation connecting each input to its normal form, and a proof that different normal forms have different meanings. Residual gates created in one normalization step must be absorbed by later steps, while every forgotten correction must be recovered at the appropriate semantic layer.

The proof problem and its solution. Our central proof-engineering question is how to certify normalization without duplicating its case analysis in a separate proof that it respects the equations. We implement the symplectic normalizer as a tower of coset tables. Each entry returns a representative, a residual word on fewer wires, and an equational certificate. Its well-definedness is proved *semantically*: the action on the bottom wire determines the representative, and completeness at the smaller width identifies the residuals. Completeness at width n is thus an input to certifying the normalizer at width $n + 1$. The well-founded dependency is the key proof-engineering result: semantic separation replaces the global comparison of table executions, while local equational certificates remain explicit.

Table 1. Relationship to the companion work. The formalization is a shared development; this paper's contribution is its checked construction and the proof architecture explained here.

Work	Responsibility
Qupit presentation	Mathematical gates, normal boxes, circuit relations, and completeness argument; announces this Agda verification.
Generic framework	Circuit congruences, normal-form transport, coset towers, and presentation composition theorems.
This paper	Semantic certification of the qupit coset tower; Pauli action compatibility; checked translations and scalar corrections; the connection to structural algebraic semantics and its certificate interface.

Only this symplectic layer needs the large normalizer. The projective semantics is the semidirect product $\mathbb{Z}_p^{2n} \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$: a Pauli correction and a symplectic transformation. We assemble its presentation from the two factors, checking their interaction at the conjugation interface. A Weyl cocycle restores the scalar phase, and a direct product supplies the sign in the original rules. The resulting proof separates normalization, semantic uniqueness, interaction laws, and changes of presentation. The expensive local circuit derivations remain necessary; composition changes where their results are used.

1.1 Contribution and overlap

This paper is the account of the formalization announced by the qupit presentation [Bian et al. 2026], whose version 2 already reports Agda verification of the scalar-sensitive rules and refers to a companion formalization manuscript. We do not claim a new mathematical rule set or a verification independent of that shared development. The generic library is also a separate contribution [Anonymous 2026b]. Table 1 makes the boundary explicit.

The methodological contribution is this division of obligations: local syntax certifies each step, semantic observations identify its head, and smaller-width completeness certifies the residual. The qupit case establishes that the division survives both a large recursive table and the recovery of information erased by quotient semantics. The technical results are:

(1) **A certified symplectic normalizer.** A doubly inductive normal form and certified coset tables give normalization. Head injectivity and completeness one width below establish well-definedness (Lemma 4.1); uniqueness and expressiveness give a presentation of $\mathrm{Sp}(2n, \mathbb{Z}_p)$ (Theorems 4.2 and 4.3).
(2) **Projective completeness by composition.** The Pauli and symplectic presentations are combined and translated to the original fifteen non-scalar rules (Theorem 5.1). The normalizer never carries Pauli corrections.
(3) **Scalar-sensitive algebraic completeness.** Checked cocycle corrections and presentation translations recover the original sixteen rules. The scalar extension is connected to the structural Weyl model, yielding a certificate interface (Theorems 5.2 and 5.3). A checked two-qutrit client replaces twelve gates by six, including a necessary scalar correction, from a computed semantic equality.

The presentation implementations check in safe Agda without postulates or holes, uniformly in width and odd prime; the multiplier rules take a primitive root and its generation proof as parameters. The semantic targets are algebraic groups. Their identification with the specified complex gate matrices, including the exact phase convention, remains outside the checked development. Table 2 names the targets, and §6 states the computational and verification limits.

Table 2. Semantic targets of the checked presentation chain. $V = \mathbb{Z}_p^{2n}$, $J_p(n)$ is the structural Weyl extension defined in §5.6. Equality of symplectic maps is extensional.

Circuit equations	Algebraic interpretation	Equality retained
Symplectic rules	$\mathrm{Sp}(2n, \mathbb{Z}_p)$	Symplectic action
Projective rules	$V \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$	Action and Pauli correction
Exact rules over ω	$J_p(n)$	Action, Pauli correction, and phase
Original rules over $-\omega$	$J_p(n) \times \mathbb{Z}_2$	The above and a scalar sign

2 Circuit Equations and the Proof Interface

The framework [Anonymous 2026b] represents a circuit language by a family of gates indexed by arity. A generator is a gate on the bottom wires or a generator shifted up one wire; a circuit of width n is a word over these generators. Words have constructors $[g]^{\mathrm{w}}$, ε, and $u \bullet v$ for a gate, identity, and composition. Wire counts in the types rule out out-of-bounds placements. Words are drawn left to right as written, with the right factor acting first. The operation $c\uparrow$ shifts every gate up one wire.

From equations to derivations. A rule set is an inductive family $\Gamma_n(u, v)$, written n SRel, u === v in the code. It contains the gate-specific equations. The library adds structural rules for shifting equations and commuting gates on disjoint wires. A zero-wire scalar also has a rule identifying it with its shift. The resulting raw relation generates a monoid congruence $\approx$, closed under reflexivity, symmetry, transitivity, composition, associativity, and the unit laws. A proof of $u \approx v$ is therefore an explicit equational derivation, not a test of semantic equality. The relation === and its congruence $\approx$ remain distinct throughout.

The presentation contract. For a group G with equality $\doteq$, a theorem Γ IsPresentationOf G supplies an interpretation $[\![_]\!]$, inverses for the presented generators, and a group isomorphism. In particular, it establishes

$$u \approx v \iff [\![u]\!] \doteq [\![v]\!], \qquad \forall x \in G.\ \exists u.\ [\![u]\!] \doteq x.$$

The forward implication is soundness, the reverse is completeness, and the last property is expressiveness. The semantic target matters: defining G to be the same word quotient would not establish a connection to an independently specified semantics.

The last two interpretations pass through presented extensions internally; Theorem 5.2 removes their word-quotient components from the semantic endpoint. No row asserts equality of complex matrices. That further correspondence must preserve the interpretation of each named gate, not just the abstract group.

Normal forms through coset tables. One way to discharge this contract is to compute a normal form nf : Circuit $n \to NF_n$ and a circuit $[u]$ for each normal form, with $c \approx [nf(c)]$. If equal denotations imply equal normal forms, soundness then gives completeness. The library constructs such normalizers one wire at a time. A coset table takes a representative c and a generator g and returns a residual word b' on fewer wires and a new representative c', together with a derivation

$$[c] \bullet [g]^{\mathrm{w}} \approx b'\uparrow \bullet [c'].$$

Iterating the table traverses a word; iterating the construction over the width builds a coset tower. The table must also respect the axioms: equivalent inputs give equal representative labels and congruent residuals. This well-definedness obligation is the key proof problem in §4.3.

Appendix A illustrates this interface for swaps. In the symplectic case the representatives encode the action on a two-dimensional wire space.

Composition and translations. The other library interface combines presentations by adding interaction laws. In a semidirect product, these move an acting generator past a normal-subgroup generator by conjugating it. In a central extension, each quotient relator additionally receives a scalar correction. A change of presentation supplies word-valued maps in both directions, preservation of axioms, and inverse laws modulo congruence. Appendix B gives the generic signatures; these constructions are dependencies rather than contributions of this paper.

Reading the Agda excerpts. Most excerpts abbreviate parameters and omit proofs; their literate sources contain hidden postulates and are scope-checked for typesetting. Check the actual safe implementations through `MainTheorems.agda`, which restates their full signatures. The new structural and client excerpts in §5.6 are also fully typechecked, without postulates. Module paths omit the prefix `Examples.Groups`.

3 Qupit Circuits and the Verification Strategy

We first specify the intended circuit language, then distinguish the semantic layers that the verification treats separately. Throughout, *the original rules* means Figure 1 of the qupit presentation [Bian et al. 2026]; figures in this paper have their own numbering.

3.1 Qupit Clifford operators

Fix an odd prime p and let $\omega = e^{2\pi i/p}$. A *qupit* is a p-dimensional quantum system with computational basis $|0\rangle, \ldots, |p-1\rangle$, indices read in $\mathbb{Z}_p$. The single-qupit Pauli operators are $X|j\rangle = |j+1\rangle$ and $Z|j\rangle = \omega^j|j\rangle$; the n-qupit Pauli group $\mathcal{P}_n$ is generated by tensor products of X and Z together with the phases ω^t. The n-qupit Clifford group C_n is generated, under composition and tensor product, by the gates of Table 3, whose qubit counterparts are shown for comparison, together with the phases ω^t. We follow the qupit paper's conventions [Bian et al. 2026]: the $\frac{1}{2}$ in S is the inverse of 2 in $\mathbb{Z}_p$, and H carries the global phase $\delta_p = i^{-(p-1)/2}$, which fixes the Fourier gate's scalar convention. This table specifies the intended matrices; the formal semantics below uses finite-field algebra instead.

Table 3. The Clifford generators for qubits and for qupits (odd prime p).

	qubit ($p = 2$)	qupit (odd prime p)
H	$\frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}$	$\lvert j\rangle \mapsto \frac{\delta_p}{\sqrt{p}} \sum_k \omega^{jk}\lvert k\rangle$
S	$\begin{pmatrix}1 & 0\\ 0 & i\end{pmatrix}$	$\lvert j\rangle \mapsto \omega^{j(j-1)/2}\lvert j\rangle$
CZ	$\mathrm{diag}(1,1,1,-1)$	$\lvert j\rangle\lvert l\rangle \mapsto \omega^{jl}\lvert j\rangle\lvert l\rangle$

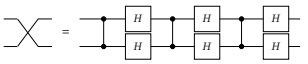
Three quotients of C_n matter. Writing $\langle\omega\rangle$ for the scalars, $\overline{C}_n = C_n/\langle\omega\rangle$ is the *projective* Clifford group and $\overline{\mathcal{P}}_n = \mathcal{P}_n/\langle\omega\rangle \cong \mathbb{Z}_p^{2n}$ the projective Pauli group, an abelian group in which X and Z commute. Conjugation by a Clifford operator permutes Paulis and preserves their commutation phases, so modulo Paulis a Clifford operator is a symplectic matrix: $C_n/\mathcal{P}_n \cong \overline{C}_n/\overline{\mathcal{P}}_n \cong \mathrm{Sp}(2n, \mathbb{Z}_p)$, the symplectic group of the $2n$-dimensional $\mathbb{Z}_p$-vector space $\mathbb{Z}_p^{2n}$ with its standard symplectic form [Bian et al. 2026], and the projective Clifford group splits: $\overline{C}_n \cong \overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. These are

the external identifications of the introduction; our formal development *defines* its semantic groups as the right-hand sides. In [Bian et al. 2026] the global phases also include -1, so that the swap and the multiplier gates belong to C_n; we first work in the projective quotient, where -1 makes no difference except that the relations come out a little simpler, and adjoin -1 only at the end (§5.5).

3.2 Circuits, rules and derivations in Agda

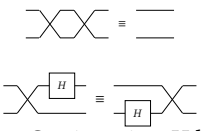
A circuit over H, S, CZ is a word in the syntax of §2. The notation $c\uparrow$ shifts a circuit up one wire; $c\downarrow$ is the identity operation, used to make the lower copy of a gate explicit. Other gates are words in this alphabet. For example, the projective swap is:

```
Ex : Word (Gen (₂₊ n))
Ex = CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑
```



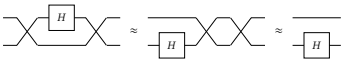
The following constructors state two projective rules: the swap is an involution, and moving H across it exchanges wires. The width is an index; the implicit n supplies any untouched wires.

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order-Ex     : (₂₊ n) SRel, Ex ^ 2 === ε
  semi-Ex-H↑ : (₂₊ n) SRel, Ex • H ↑ === H ↓ • Ex
```



A client can compose these rules into a certified local transformation. Conjugating H by the swap moves it to the lower wire:

```
conj-Ex-H↑ : Ex • H ↑ • Ex ≈ H ↓
conj-Ex-H↑ = begin
  Ex • H ↑ • Ex   ≈⟨ sym assoc ⟩
  (Ex • H ↑) • Ex ≈⟨ cleft axiom semi-Ex-H↑ ⟩
  (H ↓ • Ex) • Ex ≈⟨ assoc ⟩
  H ↓ • Ex • Ex   ≈⟨ cright axiom order-Ex ⟩
  H ↓ • ε         ≈⟨ right-unit ⟩
  H ↓ ■
```



Each step is a constructor or congruence operation in a derivation of $\approx$. The normalizer uses the same form of certificate for each coset-table entry. Completeness establishes that such certificates exist for all equal denotations in the target group.

3.3 Separating the semantic layers

The original completeness proof first normalizes circuits modulo Paulis and scalars. Its boxes yield 42 parameterized relations, reduced to 18 symplectic relations. It then lifts completeness by replacing each symplectic equation with an equation carrying a Pauli correction and pushing that correction through the remaining circuit. A similar step restores scalars [Bian et al. 2026, Proposition 4.9 and Theorem 4.10].

We keep the symplectic normal form and its relation reductions, but organize normalization as a recursive function with an equational certificate. Its well-definedness is proved using the semantic action on the bottom wire and completeness one width below. This avoids a separate global termination proof for the box-rewriting system.

For the lifting step, we use the group decomposition $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. The Pauli presentation and symplectic presentation have separate congruences; their interaction is a word-valued conjugation action. To assemble them, we prove that this action respects both congruences. These obligations follow through the semantic action, using soundness of the symplectic presentation and completeness of the Pauli presentation (§5.2). This does not eliminate the derivations of Pauli identities and

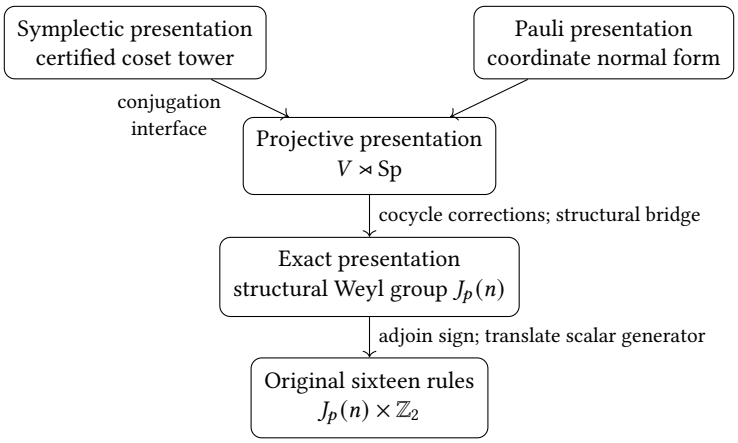


Fig. 1. The semantic dependencies. Presentation translations change the syntax at each layer without repeating the symplectic normalizer. All arrows are checked; the correspondence to complex matrices is outside the diagram.

conjugation rules: those are needed when translating the assembled presentation to H, S, CZ. It separates that syntactic work from normalization and from the generic completeness argument.

Figure 1 records the resulting dependencies. Scalar corrections are handled only after the projective presentation is established. We construct an extension from generator-level cocycle data, check every relator's correction, and then adjoin the sign and translate the scalar generators. At each stage the theorem names the specific semantic group it presents; the matrix interpretation remains outside the checked chain.

4 The Symplectic Factor

The symplectic layer is the part of the development that requires a new normalizer. We specify its semantic action, encode the original normal form, certify the coset tables, and prove uniqueness. A final relation reduction gives the smaller presentation used by the composition step.

Circuits. In the formalisation a gate set is a family of gate types indexed by arity, and a circuit on n wires is a *word* over wire-indexed generators: Circuit n = Word (Gen n) is the free monoid on the generators of H, S and CZ placed on each wire, and _↑ shifts a circuit up one wire (the generic machinery is that of §2, and the reading order of diagrams that of §3.2). The remaining usual gates are *words*, not extra generators: $Z = HHSHHS^{-1}$, $X = HSHHS^{-1}H$, the swap $Ex = (CZ \cdot H{\downarrow} \cdot H{\uparrow})^3$, and the controlled-$X$ gates $CX = H{\downarrow}^3 \cdot CZ \cdot H{\downarrow}$ and XC with the wires exchanged, where $S^{-1} = S^{p-1}$, $H^{-1} = H^3$ and $CZ^{-1} = CZ^{p-1}$ are the words that the relations make inverses, and _↓ is the identity on circuits, there to make "the lower copy of a gate" visible in a rule. One derived gate has no qubit counterpart: the *multiplier* $M_x|j\rangle = |xj\rangle$ for a unit $x \in \mathbb{Z}_p^*$, which the qupit paper spells (its Lemma 2.14) as $HS^gHS^gHS^{g^{-1}}$ followed by a Pauli correction and a global phase, and which we mostly spell $R^x\,H\,R^{x^{-1}}\,H\,R^x H$ with $R = S \cdot Z^{1/2}$, which absorbs the Pauli and drops the phase; the two spellings are interderivable (§5.3).

Parameters. Every Clifford module is parameterised by the prime, as p = p-3 + 3 with a primality proof (the symplectic modules take p = p-2 + 2, so Theorems 4.2 and 4.3 are proved for every prime), and by a primitive root g of $\mathbb{Z}_p^*$ with the witness g-gen that every unit is a power of g. So every theorem holds for every odd prime uniformly, and for the multiplicative generator chosen, exactly

as the qupit paper fixes a generator g in its Figure 1; the existence of a primitive root modulo every prime is classical and is taken as a parameter rather than proved. ($\mathbb{Z}_p$ itself is Fin p with a ring solver, Bézout inversion and Fermat's little theorem; from g and g-gen the discrete logarithm and the exponent law $g^k = g^{k \bmod (p-1)}$ are derived.)

4.1 The semantics: symplectic transformations of Pauli vectors

Rather than $2n \times 2n$ matrices, the formalisation models $\mathrm{Sp}(2n, \mathbb{Z}_p)$ as the group of $\mathbb{Z}_p$-linear, symplectic-form-preserving bijections of the phase space Pauli n = Vec $(\mathbb{Z}_p \times \mathbb{Z}_p)$ n — one pair (a, b), the exponents of $X^a Z^b$ (up to phase), per wire. A symplectic transformation is a record Symplectic n carrying the action ap, its inverse, and proofs that the two are inverse, that ap is linear, and that it preserves the symplectic form sform. Equality is extensional equality of the action alone, so the proof fields never obstruct. Composition is function composition, and therefore associativity and the unit laws hold *on the nose*, by refl — a small choice with large consequences downstream, since every monoid case of every soundness proof closes definitionally. Sp-group n is the resulting Group. The generators act as the qupit paper's conjugation tables (its Lemmas 2.24–2.25) read on exponent vectors, and [[_]] is the homomorphic extension:

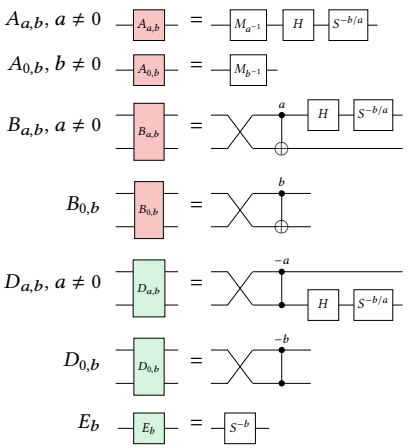
```
actg : ∀ {n} → Gen n → Pauli n → Pauli n
actg (gate₁ H-gate) ((a , b) :: ps) = (- b , a) :: ps
actg (gate₁ S-gate) ((a , b) :: ps) = (a , b + a) :: ps
actg (gate₂ CZ-gate) ((a , b) :: (a' , b') :: ps) = (a , b + a') :: (a' , b' + a) :: ps
actg (g ↑)          (x :: ps)      = x :: actg g ps
```

Composition follows the convention of §2: $w \bullet v$ applies v first, so $[\![w \bullet v]\!] = [\![w]\!] \circ [\![v]\!]$. The seven proof obligations per generator — inverses, linearity, preservation of the form — are identities in $\mathbb{Z}_p$, discharged with the ring solver. Soundness of every axiom of §4.5 in this semantics is one clause per axiom in `SoundnessDirect`.

4.2 The normal form: boxes, rows, and the staircase

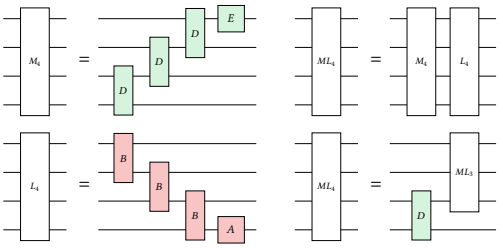
The qupit paper's symplectic normal form is a Z-normal layer followed by an X-normal layer on n wires, then a normal form on $n-1$ wires. We refine this into a doubly inductive datatype, built from four kinds of atomic boxes:

```
A B D E : Set
A = Σ[ ab ∈ (ℤ p × ℤ p) ] (ab ≢ (0 , 0))
B = ℤ p × ℤ p
D = ℤ p × ℤ p
E = ℤ p
```



The boxes are the paper's normal boxes (its Figure 6), with the same names; the pictures show how each is spelled as a circuit. Rows and the staircase are:

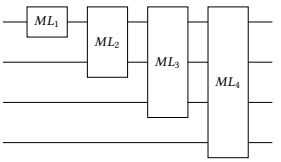
```
M L ML : ℕ → Set
M 0 = ⊤
M (1+ n) = Vec D n × E
L 0 = ⊤
L (1+ n) = Vec B n × A
ML 0 = ⊤
ML 1 = M 1 × L 1
ML m@(2+ n) = M m × L m
            ⊎ D × ML (1+ n)
```



An M-row is a column of D boxes ending in an E box (the paper's X-normal layer); an L-row is a column of B boxes ending in an A box (its Z-normal layer, with the A box at the bottom). An ML box at width $m \geq 2$ is *either* a full pair of rows *or* a bottom D box followed by an ML box one wire up: a spine of D boxes capped by a pair of rows. This is the first induction, on the width of a single coset representative. The second induction is the staircase:

```
NF : ℕ → Set
NF 0 = ⊤
NF (1+ n) = NF n × ML (1+ n)

[_] : ∀ {n} → NF n → Word (Gen n)
[_] {0}    tt        = ε
[_] {1+ n} (nf , ml) = [ nf ] ↑ • [ ml ]ml
```



Reading normal forms back as circuits is structural recursion, box by box. The box-to-word maps make the pictures above precise; for instance

```
[_]d : ∀ {n} → D → Word (Gen (2+ n))
[_]d {n} (0 , b) = Ex • CZ^ (- b)
[_]d {n} (a@(1+ _) , b) = Ex • CZ^ (- a) • H • S^ -b/a
```

where $-b/a = -b \cdot a^{-1}$, and an A box $A_{a,b}$ with $a \neq 0$ is XM $a \bullet H \bullet S^{-b/a}$, a multiplier, a Hadamard, and an S-power. Why the spine? The paper's Z-normal circuit $W_Z^{(n)}$ has its A box on some wire i, with B boxes above it and nothing below; the position i is an index that the paper's normal-form circuits carry implicitly. The ML type makes it explicit as the length of the D-spine, and, crucially, makes the coset type at width m a *sum* of the coset type at width $m-1$ and a flat case — exactly the shape the coset table and the uniqueness proof recurse on. The identity coset is the flat one with all exponents zero and $A_{0,1}$ at the bottom, and the section sends it to ε up to $\approx$, because $Ex^2 \approx \varepsilon$ collapses each $D_{0,0}$ and $B_{0,0}$.

4.3 Certifying the coset tables

The subgroup on n wires acts trivially on the bottom wire of an $(n+1)$-wire circuit. A coset representative records the remaining action on that wire's two-dimensional symplectic space. The representatives are the ML boxes, with section $[_]^{\mathsf{ml}}$. As in the swap example of Appendix A.3, a table entry returns a smaller residual circuit and a new representative:

```
ract      : ML (1+ n) → Gen (1+ n) → Circuit n × ML (1+ n)
ract-sound : ∀ (c : ML (1+ n)) g → let (b' , c') = ract c g in
             [ c ]ml • [ g ]w ≈ b' ↑ • [ c' ]ml
```

ract dispatches on the shape of the coset — a flat box or a D-spine — and on the generator — S or H on the bottom wire, CZ on the bottom two, or a lifted gate — and delegates each case to a module of the `Pushing` directory: S and H through a flat box, CZ through a flat box, CZ meeting a D box and a flat box, a lifted gate through the spine. The residual b' lives one wire up, as the

type says. The soundness proof ract-sound is a chain of $\approx$-steps per clause, each step a box relation or a structural rule. For example, an S on the bottom wire meeting a spine $D_{a,b}$ followed by lm: S commutes past the lifted lm, then the box relation $D \leftarrow S$ rewrites $D_{a,b} \bullet S$ to $dir\uparrow \bullet D_{a,b-a}$, and re-association finishes.

The generic coset construction has five obligations. Four follow by evaluation or a local derivation: the table inverts the subgroup embedding at the identity coset, the embedding respects the axioms, the identity representative denotes ε, and the table satisfies ract-sound. The fifth, h-wd-ax, requires congruent inputs to produce equal coset labels and congruent residuals. A direct symbolic comparison of table executions would repeat the large case analysis. Instead, we use the semantic action.

Lemma 4.1 (Semantic discharge of table well-definedness). *Suppose a table certifies decompositions $x \approx b\uparrow \bullet [c]$ at width $n+1$. Assume soundness at that width, that the bottom-wire action determines c, that shifting a residual preserves its semantics injectively, and completeness at width n. Then equal input denotations give equal representatives and congruent residuals.*

Proof. For two outputs, soundness gives $[[b\uparrow]]\,[[\,[c]\,]] = [[b'\uparrow]]\,[[\,[c']\,]]$. The shifted residual fixes the bottom wire, so observing that wire identifies $c = c'$. Right cancellation of the representative gives $[[b\uparrow]] = [[b'\uparrow]]$. Injectivity of the shifted interpretation gives $[[b]] = [[b']]$; completeness at width n gives $b \approx b'$. □

For h-wd-ax, the inputs differ by one axiom inside a table execution; soundness supplies their semantic equality. Head injectivity (§4.4) supplies the representative hypothesis. The shift acts as $\mathrm{id} \oplus [[b]]$, so restricting to the upper wires proves its semantic injectivity. The proof therefore requires no independent comparison of all pairs of table executions.

There is no circular appeal to completeness at the current width. The dependency is

$$\text{normalization at } n \implies \text{completeness at } n$$
$$\implies \text{table well-definedness at } n+1 \implies \text{normalization at } n+1.$$

Semantic uniqueness is proved independently of normalization, by induction on the datatype. The base normalizer at width one is built directly. The implementation packages each level and its section agreement as tower : ∀ n → TowerLevel n. Lemma 4.1 summarizes the argument used in that implementation; it is not a new generic library combinator.

The transfer criterion is concrete: a language must admit a decreasing family of sublanguages, certified decompositions, and a semantic observation that separates representatives while ignoring the residual sublanguage. Cancellation and a faithful residual embedding then reduce the obligation to completeness at the smaller instance. Here the observation is the bottom wire's action. These hypotheses explain both reuse and failure: an observation that merges distinct heads cannot justify the table, and a lossy residual embedding prevents the induction. This is a conditional proof pattern, not a claim that every normalizer admits such a decomposition. Local derivations and head injectivity remain the instance-specific work.

4.4 Uniqueness of the normal form

The uniqueness theorem states this for the symplectic semantics of §4.1:

Theorem 4.2 (unique-nf). *Normal forms with equal symplectic denotations are equal:*

unique-nf : ∀ *n* {*u v* : NF *n*} → [[*[u]*]] ≈ˢ [[*[v]*]] → *u* ≡ *v*

Home: `Symplectic.Normalization.Uniqueness`.

The action of a normal form on a Pauli vector peels the staircase one wire at a time — the coset factor acts, the resulting bottom entry is fixed, and the inner normal form acts on the tail — and injectivity is a structural induction on the staircase:

```
lemma-lm-head-inj : ∀ {n} (lm₁ lm₂ : ML (1+ n)) →
  (∀ (ps : Pauli (1+ n)) → head (act [ lm₁ ]ml ps) ≡ head (act [ lm₂ ]ml ps)) → lm₁ ≡ lm₂
lemma-lm-tail-surj : ∀ {n} (lm : ML (1+ n)) (qs : Pauli n) →
  ∃ λ (ps : Pauli (1+ n)) → tail (act [ lm ]ml ps) ≡ qs
```

Two obligations. *Tail surjectivity* is free, because a symplectic transformation carries its inverse. *Head injectivity* — the bottom-wire output determines the coset — is the crux, and it is itself an induction on the *ML* type (`LMHeadInj`): at width 1 a case analysis on the four shapes of an A box using closed forms for the actions of S^k, HS^k and the multipliers; a separation lemma showing a flat box and a spine never agree on heads; and, for two spines, the D box is read off by probing with the two basis Paulis Z and X on the bottom wire and the identity above, after which the hypothesis descends one width. With these two lemmas the generic `Circuit.Uniqueness` module of the library assembles $[[\,]]$-injective and the UniqueNormalForm witness (the uniqueness condition of §2); the module's header records the one subtlety, that ap is a *left* action — ap of a composite applies its right factor first — so the coset factor of a normal form is the one next to the input, which is what lets the coset be recovered from every state rather than from states of a restricted shape. This is the paper's Proposition 3.8 with its Lemmas 3.4–3.7, now an induction whose structure is dictated by the datatype.

4.5 Relation reduction: box relations and the two rule sets

Which relations does ract-sound use? Rewriting *every* circuit to normal form and collecting every relation the rewrite ever uses gives the paper's box relations — one per gate per box shape, parametrised by the box's labels. The formalisation organises them as nine families of case tables, each a lemma quantified over the box parameters with a computed direction dir-of and updated box:

wires	family	pushed
1	$A \leftarrow H, A \leftarrow S, E \leftarrow S$	H or S through an A box; S through an E box
2	$B \leftarrow H\uparrow, S\uparrow, S;\ D \leftarrow H, S\uparrow, S, CZ;\ L \leftarrow CZ$	a gate through a B, a D, or an L-row
3	$BB \leftarrow CZ\uparrow, B\uparrow \leftarrow CZ, DD \leftarrow CZ$	CZ through two B or two D boxes

For instance, the two-wire table for a D box states that $[d]^{\mathrm{d}} \bullet g \approx S^e \bullet dir\uparrow \bullet [d']^{\mathrm{d}}$ with e, dir and d' computed from the labels of d and the gate g — the paper's Figure 18, with the residual split into an S-power on the bottom wire and a word one wire up. The nine families are the paper's Figures 15–21; unfolding the case tables gives exactly the paper's 42 parametric relations, differing only in how cases are grouped (the formalisation, for instance, proves each of the $\uparrow/\downarrow$ pairs of a B box and a D box once and obtains the other by a duality lemma). The width-3 relations are stated at width 3 and widened to arbitrary width by a congruence $\mathrm{cong}{\downarrow^{\mathrm{k}}}$.

The box relations (BR) are derived using a calculus of *Ex*-conjugations and two-qupit lemmas from the *full* symplectic rule set, seventeen group-specific axioms whose two- and three-qupit members are Selinger's c10–c15 [Selinger 2015] verbatim and whose one-qupit members are the multiplier calculus — $S^p = H^4 = (SH)^3 = \varepsilon$, $H^2S = SH^2$, and four *schemes* over units x, y: $M_xM_y = M_{xy}$, $M_xS = S^{x^2}M_x$, and M_x on either wire past CZ gives CZ^x — with M_x spelled $S^xHS^{x^{-1}}HS^xH$. By inspection one finds a smaller set of primitive, bidirectional relations that derive all of these: the *simplified* symplectic rule set, which replaces the schemes by statements about the single multiplier M_g of the primitive root, strengthens $H^4 = \varepsilon$ to $H^2 = M_{-1}$, and drops $(SH)^3 = \varepsilon$ and $H^2S = SH^2$:

```
order-S :      (1+ n) SRel,     S ^ p === ε
order-H :      (1+ n) SRel,     H ^ 2 === M-1
M-power : ∀ k → (1+ n) SRel, Mg^ k === M (g^ k)
semi-MS :      (1+ n) SRel,     Mg • S === S^ (g * g) • Mg
semi-M↑CZ : (2+ n) SRel,     Mg ↑ • CZ === CZ^ g • Mg ↑
semi-M↓CZ : (2+ n) SRel,     Mg ↓ • CZ === CZ^ g • Mg ↓
```

plus the same order-CZ, comm-CZ-S↓, comm-CZ-S↑ and selinger-c10 to c15: fifteen group-specific rules, or eighteen with the three structural ones. The choice is not canonical; one picks the set one finds natural. The two rule sets generate the same congruence, and since they share an alphabet the identity on words is an isomorphism of presentations in both directions (`Simplified.Iso`); the substantive direction derives each full axiom from the simplified ones. The multiplier law $M_x \bullet M_y \approx M_{xy}$ is typical: write $x = g^k$, $y = g^l$, use M-power twice, add the exponents, reduce modulo $p-1$ using $M_g^{p-1} \approx \varepsilon$, which is Fermat's little theorem, and use M-power once more. This is the formal counterpart of the paper's Theorem 4.4, whose proof the paper already delegated to Agda. Read modulo Paulis and scalars, Selinger's three-qubit relations [Selinger 2015] are *identical* to selinger-c12 to c15, which is why the names were kept, and his two-qubit relations are similar; the one-qupit relations are where the odd prime shows, in the multiplier calculus of M_g with its dependence on a primitive root and on Fermat's theorem, which has no qubit counterpart. That the difficulty of the general odd prime concentrates on one wire is what makes the factored route affordable: the two- and three-wire rewriting is Selinger's, and the one-wire arithmetic is arithmetic.

4.6 The symplectic presentation theorem

Soundness (one clause per axiom), the normal form of §4.3 and its uniqueness (§4.4) assemble, through by-normalization, into a sub-presentation: the interpretation is an injective homomorphism from the words modulo the full rules into Sp-group. Surjectivity — every symplectic transformation is the action of some circuit — is proved constructively by "clear the bottom wire, recurse on the tail": the single-level input is

```
Theorem-LM : ∀ {n} (p q : Pauli n) → sform p q ≡ 1 →
  ∃ λ (lm : ML n) → act [ lm ]ml p ≡ pZ0 × act [ lm ]ml q ≡ pX0
```

— for any symplectic pair there is a coset box sending it to the bottom-wire Z and X — which is the paper's Lemma 3.7 with the box computed rather than asserted. Composing the sub-presentation of the full rules with the identity isomorphism of §4.5 gives the presentation theorem for the simplified rules, with the *same* interpretation:

Theorem 4.3 (simplified-presentation). *For every width n, the simplified symplectic rules present the symplectic group:*

```
simplified-presentation : ∀ n → (n QRel,_===_) IsPresentationOf (Sp-group n)
```

Home: `Symplectic.Simplified.Presentation`.

Here _QRel,_===_ closes the fifteen rules _SRel,_===_ of §4.5 under the three structural rules (Lift-Relation, as in §2); the statement is the index module `MainTheorems`'s. Theorem 5.1 is then built on this theorem: the semidirect product of §5.2 assembles it with a presentation of the projective Pauli group and the conjugation action into the presentation of $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$.

5 Composing the Clifford Presentations

The symplectic result supplies one factor of the projective semantics. We now add the Pauli factor, translate the resulting presentation to the original gate language, and verify the scalar corrections.

5.1 The Pauli factor

The projective Pauli group $\overline{\mathcal{P}}_n \cong (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ has an easy presentation over generators X, Z on each wire:

```
data _PRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order-X     : (1+ n) PRel, X ^ p === ε
  order-Z     : (1+ n) PRel, Z ^ p === ε
  comm-Z-X : (1+ n) PRel, Z • X === X • Z

Theorem-Pauli : ∀ {n} → (XZ._QRel,_===_ n) IsPresentationOf (+ₚ-group n)
Theorem-Pauli = XZ.presentation
```

The semantic group $+_p$-group n is Vec $(\mathbb{Z}_p \times \mathbb{Z}_p)$ n under componentwise addition — the very phase space on which $\mathrm{Sp}(2n, \mathbb{Z}_p)$ acts in §4.1, which is what will make the semidirect action definable. The normal form is $X^a Z^b$ on each wire, read off the exponent vector; soundness, uniqueness and by-normalization give the presentation, and the module also exports the soundness and completeness maps separately, which the next step consumes. (The library also presents the same group compositionally, as an n-fold direct product of $\mathbb{Z}_p \times \mathbb{Z}_p$ with no normal-form work at all; that presentation is over a different alphabet, and the circuit-alphabet one is the one the qupit chain needs.)

5.2 The semidirect product

This is the semidirect construction of §B, $\Gamma \diamond \Delta \diamond \mathrm{ConjRel}^{\mathrm{w}}\ conj$, instantiated with the Pauli presentation of §5.1 as the normal factor and the symplectic presentation of Theorem 4.3 as the acting one: the relations of both factors plus twisted commutation $h\,n\,h^{-1} = conj\,h\,n$. Here the action of a symplectic generator on a Pauli generator is the paper's Lemmas 2.24–2.25, as words — H swaps X and Z (up to an inverse), S sends X to XZ, CZ couples the two bottom wires, and a shifted gate acts on a shifted Pauli only:

```
conj : Sym.Gen n → XZ.Gen n → Word (XZ.Gen n)
conj Sym.H-gen   X-gen     = Z
conj Sym.H-gen   Z-gen     = X ^ p-1
conj Sym.S-gen   X-gen     = X • Z
conj Sym.S-gen   Z-gen     = Z
conj Sym.CZ-gen X-gen      = X • Z ↑
conj Sym.CZ-gen Z-gen      = Z
conj Sym.CZ-gen (X-gen ↑) = X ↑ • Z
conj Sym.CZ-gen (Z-gen ↑) = Z ↑
conj Sym.H-gen   (xz ↑)    = [ xz ]ʷ ↑
conj Sym.S-gen   (xz ↑)    = [ xz ]ʷ ↑
conj Sym.CZ-gen (xz ↑ ↑)  = [ xz ]ʷ ↑ ↑
conj (sym ↑)       X-gen   = X
conj (sym ↑)       Z-gen   = Z
conj (sym ↑)       (xz ↑)  = conj sym xz ↑

Pauli⋊Symplectic : (n : ℕ) → WRel (XZ.Gen n ⊎ Sym.Gen n)
Pauli⋊Symplectic n = (n PRel,_===_) ⋄ (n SpRel,_===_) ⋄ ConjRelʷ conj
```

(module qualifiers on the Pauli side elided; the relation is _QRel,_===_ in the source).

The generic theorem asks that conj respect the symplectic rules in its first argument and the Pauli rules in its second. Both go through the semantics: the action of a symplectic word c on a Pauli word w, computed syntactically by $(conj^{\mathrm{h'}})$, denotes ap $[\![c]\!]$ $(sem\ w)$ in the phase space,

so if $c \approx d$ by a symplectic axiom then the two conjugates have equal denotations by symplectic soundness, and are therefore congruent Pauli words by Pauli *completeness*:

```
respects-Δ : ∀ {n} {c d : Word (Gen n)} (u : Word (XZ.Gen n)) → Sim._QRel._===_ n c d →
             PB._≈_ (XZ._QRel._===_ n) ((conj ʰ') c u) ((conj ʰ') d u)
respects-Δ {n} {c} {d} u ax =
  complete n (Eq.trans (conjw-sem c u) (Eq.trans (denote-eq ax (sem u)) (Eq.sym (conjw-sem d u))))
```

and symmetrically for respects-Γ. The long symplectic rules — M-power, semi-M↑CZ, selinger-c10 to c15 — are never conjugated by hand. The resulting presentation theorem for Pauli⋊Sp has no further proof obligations beyond the module's prime and primitive-root parameters. The conjugation rules must still be derived from the smaller gate language in the next step; the semantic argument establishes the generic action interface without repeating those derivations for each symplectic axiom.

The semantic side. Pauli⋊Sp is the library's semidirect product of the two semantic groups, built from an Action record (§B) whose action is the syntactic conj transported through the two presentation isomorphisms. The *natural* semidirect product, with $\mathrm{Sp}(2n, \mathbb{Z}_p)$ acting on $\mathbb{Z}_p^{2n}$ by ap, is a different term, and the two are identified by a lemma act≡ap proving that the transported action is ap. This lemma lives in an abstract block, allowing later proofs to use the equality without unfolding the action through both presentation isomorphisms.

5.3 Translating back to the circuit language

To connect two presentations on different alphabets, we give word-valued generator maps in both directions, prove that they preserve the axioms, and prove that their composites are identities modulo congruence. When the alphabets agree and the maps are identities, this reduces to mutual derivability of the rule sets.

The semidirect presentation uses X, Z, H, S, CZ; the target uses H, S, CZ. Table 4 separates generator elimination from relation reduction. Eliminating X, Z requires deriving their Pauli identities and conjugation rules as equations between the corresponding H, S, CZ words. For example, the mixed equation $CZ \bullet X\uparrow \approx X\uparrow \bullet Z\downarrow \bullet CZ$ must hold after substituting the derived word for each Pauli.

Table 4. The three presentations of the projective Clifford group and the two reductions between them.

	SemiDirect	Simplified-V1	Paper-V0 / Paper-V1
generators	X, Z, H, S, CZ	H, S, CZ	H, S, CZ
relations	Pauli (3) + symplectic (15) + conjugation (14)	19	16 / 15

generator reduction: X, Z become the derived words; $S \mapsto R = S \cdot Z^{1/2}$
relation reduction: Pauli + conjugation rules become derivable; c10–c15 and the swap calculus interderivable

From the semidirect relation to Simplified-V1. The first reduction is an isomorphism of presentations on *different* alphabets, given by generator maps in both directions: f sends X and Z to their derived words, H and CZ to themselves, and S to R; h sends H and CZ to themselves and S to $Z^{-1/2} \bullet S$; both commute with $\uparrow$. The symplectic S is carried to $R = S \cdot Z^{1/2}$ and h compensates, because the target rule set, Simplified-V1, spells the multiplier over R, as §4 does: $M_x = R^x H R^{x^{-1}} H R^x H$ exactly, with no Pauli correction. Simplified-V1 has nineteen group-specific rules: the simplified symplectic rules of §4.5 with R in place of S in the multiplier rules and in c10, c11, plus $(SH)^3 = \varepsilon$, $H^2SH^2S = SH^2SH^2$, and the two rules for pushing X through CZ. Notably $XZ = ZX$ is *not* among them; it is derived from $(SH)^3 = \varepsilon$ and M-power, by the paper's own argument (its Proposition 4.8).

The isomorphism theorem needs the four generator-level facts of the library's StarGroupIsomorphism: f and h respect the axioms (f-well-defined, h-well-defined, one clause per axiom, the former containing the derivations of all three Pauli rules and all fourteen conjugation rules), and each is a left inverse of the other on generators. Composing with the semidirect theorem:

```
presentation : ∀ {n} →
  (Clifford-Relations._QRel,_===_ n) IsPresentationOf (Semidirect.Pauli⋊Sp n)
```

The generator reduction uses a multiplier calculus, a Pauli-conjugation calculus, the CZ layer, and a small rewriting tactic that discharges the bookkeeping the paper's hand derivations spend most of their length on.

Why the intermediate presentations matter. `Paper-V0` uses $R = SZ^{1/2}$ to spell multipliers and keeps $(SH)^3 = \varepsilon$ as an axiom. `Paper-V1` spells the same multipliers over S with a Pauli prefix; in that spelling the multiplier law already derives $(SH)^3 = \varepsilon$. Thus sixteen and fifteen refer to two projective axiom sets with the same congruence, not different completeness claims. The identity on words transports between them once the multiplier identity is proved. Appendix C gives the identities and the complete `Paper-V0` rule figure. None of these changes restarts the symplectic normalization proof.

Theorem 5.1 (clifford-presentation). *For every width n, the paper's fifteen rules, read modulo scalars, present the constructed projective Clifford group:*

```
clifford-presentation : ∀ n → (PapV1.Clifford-Relations._QRel,_===_ n) IsPresentationOf (Pauli⋊Sp n)
```

Home: `ProjectiveClifford.Qupit.Paper-V1.Presentation`; the same theorem holds for our sixteen rules (`Paper-V0.Presentation`).

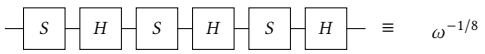
The checked statement is

$$\mathrm{Cir}\ n \,/\, \approx_{15\ \text{rules}} \xrightarrow{[\![_]\!]\ \cong} \overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p).$$

It includes soundness, completeness, and surjectivity for that target. The further identification with a quotient of a unitary group is an external mathematical result, not an additional arrow in the checked theorem. The scalar layers below first target the internal presented groups Exact-group and Clifford±; §5.6 then connects them to the structural targets of Table 2.

5.4 Restoring scalar-sensitive equations

The exact layer follows the central-extension recipe of §B: add a central generator ω of order p, and attach a scalar correction to each projective relator. The correction is trivial except for order-SH, which becomes $(SH)^3 = \omega^{(p^2-1)/8}$. The division is integral for odd p, and its residue in $\mathbb{Z}_p$ is $-1/8$:



These corrections are verified using the algebraic phase function described below. They concern `Paper-V0`'s sixteen-rule presentation. The original rules, whose multiplier definitions and scalar generator have different spellings, are reached by a further translation in §5.5.

The structural extension has a mathematical interpretation through Weyl operators and a linear Weil representation [Gross 2006; Gurevich and Hadani 2007]. Appendix D records this external argument. Matching the named gates to their specified matrices and phases is a separate obligation; none of the presentation theorems below assumes that match has been verified.

The semantic side. A central extension of a group Q by an abelian group K is determined by a 2-cocycle $c : Q \times Q \to K$, with multiplication $(k, g) \cdot (k', h) = (k + k' + c(g, h),\ gh)$. The formalisation builds two. The *structural* one is the Weyl cocycle $c((P, S), (Q, _)) = \frac{1}{2}\,\mathrm{sform}(P, \mathrm{ap}\ S\ Q)$ on the natural semidirect product, whose cocycle law is symplectic invariance of sform, and which satisfies the Heisenberg law $P \cdot Q = \omega^{\mathrm{sform}(P,Q)}\ Q \cdot P$ on Paulis — what distinguishes it from the direct product $\langle\omega\rangle \times (\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p))$. The *presented* one, Presented-group, has as quotient the word group of the sixteen rules and as kernel the word group of $\langle \omega \mid \omega^p \rangle$, and its cocycle is generated from *generator data*: for each gate a and word v, the scalar by which the lifts of a and v fail to compose, together with the proof that this descends to the quotient and assigns each relator the same correction on both sides. The generator data is obtained by pulling the Weyl cocycle back along the projective presentation theorem, so the generated cocycle agrees with that pullback by the checked lemma agree. Section 5.6 uses this agreement to build an isomorphism of the total groups. With the prime and primitive-root parameters fixed, the theorem is

```
presentation-exact : ∀ n → (n Exact,_===_) IsPresentationOf (PE.Presented-group n (exact-generator-data n))
```

Its proof needs no normal forms — since both factors are word groups, [[_]] is a splitting rather than a normalisation — but it does need, for each of the sixteen rules, that the phase the Weyl cocycle assigns to the left-hand side equals the correction plus the phase of the right-hand side. This reduces to nineteen equations in $\mathbb{Z}_p$ about an explicit phase function on circuits, discharged by Pauli arithmetic, by a one-wire calculus that reads a circuit as a triple (Pauli, 2×2 symplectic matrix, phase) with an explicit concatenation law, and, for order-SH, by the number-theoretic fact that the residue of $(p^2 - 1)/8$ is $-1/8$. The lemma split-forces checks that the trivial cocycle with trivial gate lifts would force this correction to be trivial; it is not a theorem ruling out every possible splitting of the extension. This is the formal counterpart of the paper's remark before its Theorem 4.10 that a complete rule set for $\langle\omega\rangle$ "only needs $\omega^d = 1$", with the one non-trivial correction computed and checked.

5.5 The paper's Figure 1, scalars included

The scalar kernel of the exact group in §5.4 has odd order p, so it contains no scalar sign of order two. The paper's Figure 1, however, is stated over $-\omega$, H, S, CZ, and its derived generators need -1: the multiplier M_a (its (T1)) carries the Legendre symbol $(\frac{a}{p}) = \pm 1$ besides a power of ω, and the swap (T4) carries $\lambda_p^2 = (-1)^{(p-1)/2}$. Since 2 and p are coprime, $-\omega$ generates the same scalars as -1 and ω together, and -1 is central and meets the exact group trivially, so the paper's group is the direct product of the exact group with $\langle -1 \rangle \cong \mathbb{Z}_2$. Syntactically that is the join of §B: the rule set _Exact±,_===_ is the exact one $\oplus$ the cyclic relation $(-1)^2 = \varepsilon$, and the direct-product theorem gives clifford±-presentation onto Clifford± n, with the same module parameters.

Figure 1 itself is formalised as printed: $-\omega$ is a gate on no wires, ν, whose centrality is the theorem comm_0 of the structural rules; $\omega = \nu^{p+1}$ and $-1 = \nu^p$; the derived generators (T1)–(T7) keep their phases, the Legendre symbol computed by Euler's criterion $a^{(p-1)/2}$; and the rules are C0–C15, C0 being $\nu^{2p} = \varepsilon$. The two rule sets are isomorphic by the exchange of scalar generators, $\nu \mapsto -1 \cdot \omega$ one way and $\omega \mapsto \nu^{p+1}$, $-1 \mapsto \nu^p$ the other, the identity on gates. Each direction must respect the other's rules. That Figure 1's rules hold in the rule set with -1 is where all its scalars are checked, once: the gate part of each multiplier is the S-spelled multiplier of `Paper-V1`, and the phase (T1) attaches to it is exactly the inverse of the phase the Weyl cocycle assigns that gate part, so that the Legendre symbol is all that is left, and it is a character along the powers of g with the right value at -1. The converse — each exact rule, with its power of ω, holds in Figure 1 — computes no scalar inside Figure 1 at all. Modulo scalars the rule is a theorem of Figure 1, which with its scalars erased is isomorphic to `Paper-V1` (its C4 conjugates by M_g where `Paper-V1`'s semi-MR conjugates

by M_g^{-1}, and each derives the other); so it holds in Figure 1 up to some power $(-\omega)^a$. Read back by the isomorphism, that equation holds in the rule set with -1, as does the rule itself, so $(-\omega)^a$ equals the rule's own scalar there; and the presentation theorem reads a power of $-\omega$ off as its exponents modulo p and modulo 2, which pins a down and rebuilds the scalar in Figure 1.

The existing presentation-Figure1 theorem therefore presents the internal group Clifford±. Its interpretation is the checked translation of $-\omega$ to $-1 \cdot \omega$ followed by the presentation with the sign adjoined. The next step removes the remaining word-group components from that target.

5.6 Structural semantics and a certificate client

Write $J_p(n)$ for the structural Weyl extension. Its elements are $(e, P, S) \in \mathbb{Z}_p \times \mathbb{Z}_p^{2n} \times \mathrm{Sp}(2n, \mathbb{Z}_p)$, with multiplication

$$(e, P, S)(f, Q, T) = \big(e + f + \tfrac{1}{2}\mathrm{sform}(P, SQ),\ P + SQ,\ ST\big).$$

This definition contains no circuit congruence. To connect it to Exact-group, define $\chi(a, u) = (\mathrm{read}(a), [\![u]\!]_{\mathrm{proj}})$, where a is a scalar word and u a projective gate word.

Theorem 5.2 (Structural scalar presentation). *For every width n, χ is a group isomorphism from the presented scalar extension to $J_p(n)$. Composing it with the exact presentation gives a presentation of $J_p(n)$ with the specified interpretation.*

Proof. The scalar presentation reads ω^k as $k \bmod p$, and the projective presentation reads u as its Pauli and symplectic components. The generated cocycle is congruent to the scalar-word embedding of the pulled-back Weyl cocycle (SemGeneratorData.agree). Reading this equation in $\mathbb{Z}_p$ proves that χ preserves the twisted multiplication. Injectivity and surjectivity follow componentwise from the two factor presentations. This also fixes the interpretation: scalar words retain their exponents, and gate words retain their projective interpretation and accumulated Weyl phase. □

```
exact-structural-presentation : ∀ n →
  (n Exact._===_) IsPresentationOf (Model.Clifford-group n)
exact-structural-presentation = Structural.Bridge.structural-presentation
```

Theorem 5.3 (Original scalar-sensitive rules). *For every width n, the original sixteen rules over $-\omega, H, S, CZ$ present $J_p(n) \times \mathbb{Z}_2$.*

The map is χ on the exact component and the identity on the sign. Composing with the checked Figure 1 translation gives:

```
figure1-structural-presentation : ∀ n →
  (n F._===_) IsPresentationOf (StructuralClifford± n)
figure1-structural-presentation = Signed.At.presentation
```

These excerpts are aliases of the actual implementations, with no postulated abbreviations; their literate module is also fully typechecked. The parameters are the odd prime and primitive-root witness of §4. The result strengthens the semantic endpoint, but does not assert that the named generators denote the intended complex matrices.

Repairing a projectively valid replacement. Suppose a client proposes to replace u by v and proves $[\![u]\!]_{\mathrm{proj}} = [\![v]\!]_{\mathrm{proj}}$. This premise includes the Pauli component: agreement of symplectic matrices alone is insufficient. The Weyl evaluator computes the scalar phase by

$$\Phi(\varepsilon) = \Phi([a]) = 0, \qquad \Phi(uv) = \Phi(u) + \Phi(v) + c([\![u]\!]_{\mathrm{proj}}, [\![v]\!]_{\mathrm{proj}}).$$

Define $t = \Phi(u) - \Phi(v)$ in $\mathbb{Z}_p$. The client returns the exact derivation

$$[u]_{\text{gate}} \approx_{\text{exact}} [\omega^t]_{\text{scalar}} \bullet [v]_{\text{gate}}.$$

Its implementation reads both sides in $J_p(n)$, proves equality by finite-field arithmetic and the supplied projective equality, and applies the presentation's injectivity. It performs no search through the circuit equations.

```
phase-repair : ∀ n (u v : Syn.Circuit n) →
  Group._≈_ (Model.Pauli⋊Sp-group n)
    (FE.Weyl.[[_]] n u) (FE.Weyl.[[_]] n v) →
  PB._≈_ (n Exact,_===_) [ u ]ᵣ
    ([ Sc.emb (Client.At.correction n u v) ]ₗ • [ v ]ᵣ)
phase-repair = Client.At.repair
```

A checked two-qutrit replacement. At $p = 3$, let H_i, S_i act on wire i, with wire zero at the bottom, and write $C = CZ_{0,1}$. Products use the circuit syntax's order. Consider

$$u = S_1 C H_1 H_0 H_1 C S_1 S_0 H_0 C S_0 H_0,$$
$$v = S_0 S_0 S_1 H_1 H_1 C.$$

The module `Qupit-Clifford.CertificateExample` supplies the complete semantic premise and calls repair. A compact evaluator computes the Pauli vector, symplectic action, and Weyl phase; a structural induction proves agreement with the presentation's interpretation. Agda checks equality of the Pauli vectors, checks the actions on all $3^4 = 81$ phase-space vectors, and computes $\Phi(u) = 1$, $\Phi(v) = 0$. The result is the exact certificate $[u]_{\text{gate}} \approx_{\text{exact}} [\omega]_{\text{scalar}} \bullet [v]_{\text{gate}}$, which replace lifts into arbitrary surrounding words.

The gate counts are also checked: twelve H, S, CZ gates become six, and three entangling gates become one. The replacement retains one explicit scalar gate; the counts exclude that scalar. Deleting it would lose the exact phase. An external bounded search found this candidate; `notes/find-certificate-example.py` reproduces the search. Its output is untrusted: the Agda module separately verifies the equality and constructs the derivation. This example exercises semantic checking, phase repair, and contextual replacement together; it is not a benchmark or an optimality result.

For the original gate language, Figure1.Structural.certify takes equality in $J_p(n) \times \mathbb{Z}_2$ and returns a derivation in its sixteen rules. Its replace operation lifts that derivation into any surrounding words l, r. These interfaces demonstrate certificate construction, not equivalence discovery: the caller must supply the semantic equality proof. There is no packaged full-circuit decision procedure, optimizer, or claim about certificate size.

6 Validation, Proof Engineering, and Limits

6.1 What the checked statements establish

`MainTheorems.agda` restates the main results with full signatures and imports their implementations, including the structural scalar isomorphism, the phase-repair client, the symmetric-group example, and structural-rule independence. The implementation checks with Agda 2.8.0 and standard library 2.4 under `--safe`, without postulates or holes. The trust base includes Agda's implementation and the checked standard library. No external algebra system supplies unchecked results. Most literate excerpts are only scope-checked; the new structural presentation and client excerpts are also fully typechecked aliases.

All four semantic endpoints in Table 2 are now built from finite-field data and symplectic transformations. The exact proof still uses word groups internally, but the checked map χ removes

Table 5. Obligations discharged at interfaces and obligations retained.

Interface	Discharged by the architecture	Still proved for qupits
Coset table	Compatibility follows from semantic separation and completeness one width below	Local pushing certificates and semantic head injectivity
Pauli composition	Completeness follows from the two factor presentations and action compatibility	Pauli conjugation laws and their translations into H, S, CZ
Scalar extension	Completeness follows from word splitting; the structural bridge uses factor isomorphisms	Cocycle laws and each relator's phase correction
Presentation change	Axiom preservation and inverse maps transport completeness	Derivations of the target rules and the multiplier spelling identity

them from the target. Its definition fixes how scalars and projective gates are read; it is not just an existence claim about isomorphic groups.

The remaining semantic obligation is a commuting interpretation into complex matrices: each generator must map to the intended matrix with its specified phase. The representation argument in Appendix D provides external mathematical context, not this checked correspondence. In particular, the structural bridge does not by itself certify a compiler whose specification is matrix equality. The multiplier rules also take a primitive root and a proof that its powers cover the nonzero field; the artifact does not construct that witness for every prime.

6.2 Which obligations composition changes

Table 5 assesses the method at the level of actual proof obligations. It does not estimate how long an alternative implementation would take.

The multiplier change illustrates the separation concretely. Replacing the R-spelling by the original S-spelling requires a Pauli-prefixed multiplier identity and checking the affected axioms (§5.3); it does not alter the coset tables or their uniqueness proof. Conversely, changing the symplectic normal boxes would require new local certificates and a new head-injectivity argument. Decomposition does not make those obligations disappear.

The scalar layer also exposes a limit of generic reuse. The general extension theorem expects bijective normal forms for its factors. The projective presentation here is not exported in that form, so the exact proof establishes completeness by splitting scalar and gate words instead. The structural bridge subsequently uses the two factor isomorphisms and cocycle agreement, without introducing another normalizer.

6.3 Execution checks and reproducibility

The symplectic normalizer is executable. The module `Symplectic.Normalization.Computations` checks nine closed equalities by reduction alone: normalization of S^p and H^4 at width one, and of CZ^p at width two, for each $p = 3, 5, 7$, agrees with normalization of the empty circuit. Each proof is reflexivity after evaluation, not an application of the corresponding presentation axiom. These are small execution checks, not a circuit benchmark suite.

The two-qutrit client (§5.6) additionally checks a replacement involving both wires and multiple entangling gates, including its semantic premise and scalar correction. Its compact evaluator is proved correct for every word over the five two-wire gate positions; the equality check used here

enumerates phase-space vectors. We report no scalable checker, optimizer integration, or bound on generated derivation size. The example demonstrates the certificate contract; it does not measure general optimization quality.

A fresh project check completed in one process in 442.9 seconds, with a peak resident set of 3.66 GiB, on a Core i5-13400TEF under WSL2 using Agda 2.8.0 and a 5 GiB compacting heap. Project sources were copied to the Linux temporary filesystem without interfaces; installed standard-library 2.4 interfaces were retained. This is a cold project check, not a cold build of every dependency. The script `notes/check-artifact.py` reproduces the procedure; `notes/validation/fresh-check.json` records the source fingerprint, environment, exit status, elapsed time, and peak memory. The measurement concerns proof checking, not circuit execution or optimization.

The retained artifact contains 247 Agda files and 76 439 lines, including comments and blank lines. About 40% is in the symplectic layer and 21% in projective composition and translations. Appendix E gives the breakdown and counting procedure. These counts describe source distribution, not saved effort or execution cost.

Managing definitional computation. Unfolding semantic definitions through presentation translations can expand large proof terms during conversion checking. The phase evaluator therefore exposes defining equations, and the structural bridge uses cocycle agreement. Explicit endpoints for extensional equality prevent unification from reconstructing symplectic records. These choices affect type checking, not the trust boundary.

6.4 AI Use

Versions of Claude assisted the Agda formalization, including composition of box relations for ract-sound, the symplectic-group definition, and equivalence with the original rules modulo scalars. Claude also produced the initial presentation-exact proof and the Figure 1 layer with −1. These contributions were checked and corrected by the authors. Claude assisted the earlier manuscript draft. OpenAI Codex assisted the PLDI revision, source inspection, and validation, and implemented the structural scalar bridge, its sign extension, the phase-repair client, the closed execution checks, and the two-qutrit certificate example with its discovery script added in this revision. Agda checks the resulting proof terms; it does not check that their statements match the intended mathematics or that the prose accurately describes their scope.

7 Related Work

Verified circuit transformations and optimization. SQIR/VOQC [Hietala et al. 2021] verifies quantum circuit transformations against complex-matrix semantics. QWIRE [Paykin et al. 2017], CoqQ [Zhou et al. 2023], and Qbricks [Chareton et al. 2021] provide languages and infrastructure for reasoning about quantum programs. Quartz [Xu et al. 2022] generates candidate transformations by enumerating small circuits, verifies them with an automated theorem prover, and applies them in a cost-based search. Our presentation theorems address whether a fixed family of equations captures all equalities in the algebraic semantics. The certificate client demonstrates one use of that contract; we do not evaluate an optimizer integration or inherit these systems' matrix interpretation.

The qupit presentation and its predecessors. The closest work is the qupit presentation of Bian et al. [Bian et al. 2026]. Version 2 reports Agda verification of the scalar-sensitive rules and refers to a companion manuscript for the formalization. Table 1 distinguishes that shared development from the generic library [Anonymous 2026b] and the proof architecture presented here. The present contribution concerns how the library's hypotheses are discharged: semantic separation and smaller-width completeness certify the recursive tables, and explicit compatibility proofs

connect the layers to structural semantics. The mathematical rule set and the composition theorems themselves belong to the companion work.

The normal-form method descends from Selinger's qubit Clifford presentation [Selinger 2015] and its qutrit generalization [Li et al. 2025]. Other gate-group presentations include CNOT-dihedral [Amy et al. 2018], real stabilizer [Makary et al. 2021], two-qubit Clifford+T [Bian and Selinger 2023a], three-qubit Clifford+CS [Bian and Selinger 2023b], and Toffoli–Hadamard circuits [Amy et al. 2024]. Bian and Selinger's Agda developments check equivalences between syntactic presentations. Here those translations are one part of a chain that also establishes a presentation of independently constructed projective and scalar semantics.

Complete circuit and graphical calculi. Clément et al. give complete equational theories for arbitrary quantum circuits [Clément et al. 2023], with extensions for ancillae and discarding [Clément et al. 2024]. Blake's polycontrolled PROPs give a finite schematic theory for exact qudit circuits in arbitrary finite dimension [Blake 2026a]; his phase-affine calculi obtain completeness through normal forms [Blake 2026b]. These languages and rule sets have different primitives and scope from the Clifford gate language considered here. The ZX-calculus has completeness results for stabilizer fragments and Clifford+T [Backens 2014; Booth and Carette 2022; Jeandel et al. 2018; Poór et al. 2023]. A graphical completeness proof can use intermediate diagrams outside the unitary circuit language; obtaining derivations entirely within a fixed circuit gate set requires an additional argument. VyZX [Lehmann et al. 2026] supplies formal infrastructure for graphical quantum reasoning.

Mechanized algebra and reversible languages. Choudhury, Karwowski, and Sabry [Choudhury et al. 2022] mechanize the reversible language Π as a free symmetric rig groupoid. Carette et al. [Carette et al. 2024] extend this setting toward quantum computation, including a mechanized two-qubit Clifford completeness result. Their categorical semantics and our fixed-width group presentations expose different interfaces for equational reasoning. General-purpose algebra libraries provide a further foundation: mathlib has presented groups [The mathlib Community 2020, 2026], Isabelle developments formalize free groups and Nielsen–Schreier [Breitner 2010; Kharim et al. 2023], and MathComp supports large finite-group proofs [Gonthier et al. 2013]. Our use of coset tables specializes classical computational group theory [Holt et al. 2005; Sims 1994] to a supplied circuit relation set, with every table entry justified in its generated congruence and the semantic uniqueness proof supplied separately.

8 Conclusion

Completeness at a smaller width can certify the next normalization step when a semantic observation separates its representatives. The qupit development realizes this dependency in a certified coset tower, then restores Pauli and scalar information by composition. Its structural interpretation also supports a checked client: a nontrivial circuit replacement receives an exact equational certificate and its required phase correction. The proof architecture separates local derivations, semantic uniqueness, and compatibility obligations. A gate-preserving complex-matrix interpretation and a general search procedure remain outside the checked result.

Data-Availability Statement

The artifact is an Agda development containing the theorem implementations, their dependencies, the literate paper sources, and build instructions. `MainTheorems.agda` is the entry point for checking the results; it requires Agda 2.8.0 and standard library 2.4. The artifact's scope is the algebraic presentation theorems described in §6; it does not include a verified unitary-matrix interpretation or compiler performance experiments. The repository link is omitted for double-blind review. We intend to submit the artifact for PLDI artifact evaluation if the paper is accepted.

References

Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A Finite Presentation of CNOT-Dihedral Operators. *Electronic Proceedings in Theoretical Computer Science* 266 (2018), 84–97.

Matthew Amy, Neil J. Ross, and Scott Wesley. 2024. A Sound and Complete Equational Theory for 3-Qubit Toffoli-Hadamard Circuits. *Electronic Proceedings in Theoretical Computer Science* 406 (2024), 1–43. arXiv:2407.11152.

Anonymous. 2026a. An Agda library for normal forms and complete relations for circuits. Supplementary material.

Anonymous. 2026b. Normal Forms and Complete Relations for Circuits in Agda. Under submission; provided as supplementary material.

Miriam Backens. 2014. The ZX-calculus is complete for stabilizer quantum mechanics. *New Journal of Physics* 16 (2014), 093021. arXiv:1307.7025.

Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A Complete and Natural Rule Set for Multi-Qudit Clifford Circuits in All Odd Prime Dimensions. arXiv:2609.40106 https://arxiv.org/abs/2609.40106v2 Version 2, 5 October 2026.

Xiaoning Bian and Peter Selinger. 2023a. Generators and Relations for 2-Qubit Clifford+T Operators. *Electronic Proceedings in Theoretical Computer Science* 394 (2023), 13–28. arXiv:2204.02217.

Xiaoning Bian and Peter Selinger. 2023b. Generators and Relations for 3-Qubit Clifford+CS Operators. *Electronic Proceedings in Theoretical Computer Science* 384 (2023), 114–126. arXiv:2306.08530.

Colin Blake. 2026a. A Complete Equational Presentation of Qudit Circuits via Polycontrolled PROPs. In *51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026) (LIPIcs, Vol. 386)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:18. doi:10.4230/LIPIcs.MFCS.2026.6

Colin Blake. 2026b. Completeness for Prime-Dimensional Phase-Affine Circuits. arXiv:2603.06466.

Robert I. Booth and Titouan Carette. 2022. Complete ZX-Calculi for the Stabiliser Fragment in Odd Prime Dimensions. In *47th International Symposium on Mathematical Foundations of Computer Science (MFCS 2022) (LIPIcs, Vol. 241)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 24:1–24:15. doi:10.4230/LIPIcs.MFCS.2022.24 arXiv:2204.12531.

Joachim Breitner. 2010. Free Groups. Archive of Formal Proofs, https://www.isa-afp.org/entries/Free-Groups.html.

Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a Few Square Roots, Quantum Computing Is as Easy as Π. *Proceedings of the ACM on Programming Languages* 8, POPL, Article 19 (2024), 29 pages. doi:10.1145/3632861 arXiv:2310.14056.

Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An Automated Deductive Verification Framework for Circuit-Building Quantum Programs. In *Programming Languages and Systems: 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. Springer, Cham, Switzerland, 148–177. doi:10.1007/978-3-030-72019-3_6

Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in Reversible Programming: From Symmetric Rig Groupoids to Reversible Programming Languages. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 6:1–6:32.

Alexandre Clément, Noé Delorme, Simon Perdrix, and Renaud Vilmart. 2024. Quantum Circuit Completeness: Extensions and Simplifications. In *32nd EACSL Annual Conference on Computer Science Logic (CSL 2024) (LIPIcs, Vol. 288)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 20:1–20:23. doi:10.4230/LIPIcs.CSL.2024.20

Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2023. A Complete Equational Theory for Quantum Circuits. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. IEEE, Piscataway, NJ, USA, 1–13. doi:10.1109/LICS56636.2023.10175801 arXiv:2206.10577.

Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Pasca, Laurence Rideau, Alexey Solovyev, Enrico Tassi, and Laurent Théry. 2013. A Machine-Checked Proof of the Odd Order Theorem. In *Interactive Theorem Proving (ITP 2013) (Lecture Notes in Computer Science, Vol. 7998)*. Springer, Berlin, Heidelberg, 163–179. doi:10.1007/978-3-642-39634-2_14

David Gross. 2006. Hudson's theorem for finite-dimensional quantum systems. *J. Math. Phys.* 47, 12 (2006), 122107. doi:10.1063/1.2393152

Shamgar Gurevich and Ronny Hadani. 2007. The geometric Weil representation. *Selecta Mathematica* 13 (2007), 465–481. doi:10.1007/s00029-007-0047-3

Shamgar Gurevich and Ronny Hadani. 2010. Bounds on certain higher-dimensional exponential sums via the self-reducibility of the Weil representation. arXiv:math/0612765 [math.RT] https://arxiv.org/abs/math/0612765

Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A Verified Optimizer for Quantum Circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (2021), 29 pages. doi:10.1145/3434318

Derek F. Holt, Bettina Eick, and Eamonn A. O'Brien. 2005. *Handbook of Computational Group Theory.* Chapman & Hall/CRC, Boca Raton, FL, USA.

Emmanuel Jeandel, Simon Perdrix, and Renaud Vilmart. 2018. A Complete Axiomatisation of the ZX-Calculus for Clifford+T Quantum Mechanics. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2018).* Association for Computing Machinery, New York, NY, USA, 559–568. doi:10.1145/3209108.3209131

Aabid Seeyal Abdul Kharim, T. V. H. Prathamesh, Shweta Rajiv, and Rishi Vyas. 2023. Formalizing Free Groups in Isabelle/HOL: The Nielsen-Schreier Theorem and the Conjugacy Problem. In *Intelligent Computer Mathematics (CICM 2023) (Lecture Notes in Computer Science, Vol. 14101).* Springer, Cham, Switzerland, 158–173. doi:10.1007/978-3-031-42753-4_11

Adrian Lehmann, Ben Caldwell, Bhakti Shah, William Spencer, and Robert Rand. 2026. VyZX: Formal Verification of a Graphical Quantum Language. https://arxiv.org/abs/2311.11571v4 arXiv:2311.11571, version 4, 8 April 2026.

Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A Complete and Natural Rule Set for Multi-Qutrit Clifford Circuits. *Electronic Proceedings in Theoretical Computer Science* 426 (2025), 23–78. arXiv:2508.14670.

Justin Makary, Neil J. Ross, and Peter Selinger. 2021. Generators and Relations for Real Stabilizer Operators. *Electronic Proceedings in Theoretical Computer Science* 343 (2021), 14–36. arXiv:2109.05655.

Markus Neuhauser. 2002. An Explicit Construction of the Metaplectic Representation over a Finite Field. *Journal of Lie Theory* 12, 1 (2002), 15–30. doi:10.5802/jolt.250

Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017).* Association for Computing Machinery, New York, NY, USA, 846–858. doi:10.1145/3009837.3009894

Boldizsár Poór, Robert I. Booth, Titouan Carette, John van de Wetering, and Lia Yeh. 2023. The Qupit Stabiliser ZX-travaganza: Simplified Axioms, Normal Forms and Graph-Theoretic Simplification. *Electronic Proceedings in Theoretical Computer Science* 384 (2023), 220–264. doi:10.4204/EPTCS.384.13 arXiv:2306.05204.

Peter Selinger. 2015. Generators and relations for n-qubit Clifford operators. *Logical Methods in Computer Science* 11, 2:10 (2015), 1–17. doi:10.2168/LMCS-11(2:10)2015 arXiv:1310.6813.

Charles C. Sims. 1994. *Computation with Finitely Presented Groups.* Encyclopedia of Mathematics and its Applications, Vol. 48. Cambridge University Press, Cambridge, UK. doi:10.1017/CBO9780511574702

The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020).* Association for Computing Machinery, New York, NY, USA, 367–381. doi:10.1145/3372885.3373824

The mathlib Community. 2026. Mathlib documentation: PresentedGroup, CoxeterGroup, and the Nielsen-Schreier theorem. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/FreeGroup/NielsenSchreier.html.

Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A. Acar, and Zhihao Jia. 2022. Quartz: Superoptimization of Quantum Circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022).* Association for Computing Machinery, New York, NY, USA, 625–640. doi:10.1145/3519939.3523433

Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational Verification of Quantum Programs. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 29 (2023), 33 pages. doi:10.1145/3571222

A Worked Example: Permutation Circuits

This appendix expands the interface of §2 on circuits generated by the two-wire swap σ. The example, from the presentation library [Anonymous 2026a,b], exhibits the same progression used for the symplectic factor: a circuit syntax, relations, a coset tower, a semantic uniqueness proof, and a presentation theorem for the symmetric groups.

A.1 Gates, generators, and circuits

A gate set is a family $\mathsf{Gate} : \mathbb{N} \to \mathsf{Set}$ giving, for each arity, the gates on that many adjacent wires; for permutations there is one gate, on two wires. The generic circuit layer of the library turns any such family into wire-indexed *generators*: a gate placed at the bottom of the wires, or a generator shifted up by one wire.

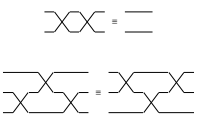
```
data Gate : ℕ → Set where
  σ-gate : Gate 2
data Gen : ℕ → Set where
  gate₁ : Gate 1 → Gen (1+ n) - a 1-ary gate on the bottom wire
  gate₂ : Gate 2 → Gen (2+ n) - a 2-ary gate on the bottom two wires
  _↑ : Gen n → Gen (1+ n)     - the same generator, one wire higher
```

Here $_{1+}\ n$ and $_{2+}\ n$ are the library's patterns for $1 + n$ and $2 + n$, and n is an implicit argument, so $\mathsf{gate}_2\ \sigma\text{-gate}$ is a generator at any width from two upward. An n-wire *circuit* is a word of generators, $\mathsf{Circuit}\ n = \mathsf{Word}\ (\mathsf{Gen}\ n)$: a single generator $[\, g \,]^{\mathsf{w}}$, the empty circuit ε — n bare wires, the identity circuit at that width — or a sequential composition $c \bullet c'$. We write σ for the one-letter circuit $[\, \mathsf{gate}_2\ \sigma\text{-gate} \,]^{\mathsf{w}}$ and $c\uparrow$ for c shifted up one wire ($_\uparrow = \mathsf{wmap}\ _\!\uparrow$). Read diagrammatically, with wires drawn bottom-up and a word drawn left to right as written, σ is a crossing of the bottom two wires, $\sigma\uparrow$ the same crossing one wire higher, and $\sigma \bullet \sigma\uparrow \bullet \sigma$ the three-wire braid on the left of the yang-baxter equation drawn beside its axiom below.

A.2 Relations: two axioms, structure for free

The group-specific relations are an inductive family indexed by the wire count. Each axiom is stated once, with its gates at the bottom of the wires and the width left open (the implicit n), so that one constructor covers every width at which the axiom fits; the structural rule cong↑ below moves it to any height. (Agda names may be mixfix: each underscore in _SRel,_===_ marks an argument position, so the family is applied as n SRel, w === v, read "$w = v$ is an axiom on n wires".)

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order : (2+ n) SRel, σ • σ === ε
  yang-baxter : (3+ n) SRel, σ • σ ↑ • σ === σ ↑ • σ • σ ↑
```



These two axioms are two thirds of the Coxeter presentation of S_n: $\sigma^2 = \varepsilon$ and the braid (Yang–Baxter) relation. The third Coxeter relation, that swaps on disjoint wires commute, is not specific to permutations: it is one of the relations every circuit theory needs, whatever its gates. By a *circuit theory* we mean a gate set with gate-specific relations, presenting for each width n a monoid of n-wire circuits under sequential composition. Such circuits are the fixed-width slices of a symmetric monoidal category (a PROP), and the monoidal structure leaves exactly two traces at fixed width: the shift $c \mapsto c\uparrow$ is "tensoring with an identity wire", so equations must be preserved by it, and the interchange law says that gates on disjoint wires commute. (The library also admits 0-ary gates — global scalars, available at every width — with one more structural rule, that a scalar is identified with its own shift, $\omega\uparrow = \omega$; that it then commutes with every generator is a theorem, once scalars

are assumed to commute with one another, which is vacuous for a single scalar.) These are the structural rules every circuit theory shares — what a pen-and-paper presentation leaves implicit in "a congruence compatible with composition and tensor product" — and the circuit layer adds them generically: Lift-Relation extends any gate-specific family to the full relation set _VRel,_===_:

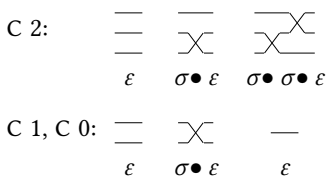
```
data _VRel,_===_ : (n : ℕ) → CRel n where
  srel : n SRel, w === v → n VRel, w === v
  cong↑ : n VRel, w === v → (1+ n) VRel, w ↑ === v ↑
  comm₁ : (h : Gate 1) (g : Gen n) → (1+ n) VRel, [ g ↕ ]ʷ • [ gate₁ h ]ʷ === [ gate₁ h ]ʷ • [ g ↕ ]ʷ
  comm₂ : (h : Gate 2) (g : Gen n) → (2+ n) VRel, [ g ↕ ↕ ]ʷ • [ gate₂ h ]ʷ === [ gate₂ h ]ʷ • [ g ↕ ↕ ]ʷ
```

From here on, ≈ denotes the monoid congruence generated by n VRel,_===_: the least relation containing the axioms and closed under reflexivity, symmetry, transitivity, congruence of •, associativity, and the unit laws; deciding it is the word problem. Because σ is its own inverse and shifts of invertible generators are invertible, the presented monoid is a group; we witness this by a Grouplike proof (every generator has a left inverse), by induction on generators.

A.3 Cosets and the staircase normal form

The engine behind the normal form is coset enumeration along the chain $S_1 < S_2 < \cdots < S_n < S_{n+1}$, where S_n sits inside S_{n+1} as the permutations fixing the bottom wire — syntactically, the circuits of the form $c \uparrow$. Read as permutations, two circuits lie in the same right coset of S_n exactly when they bring the same wire to the bottom position; there are $n{+}1$ choices, so S_n has index $n{+}1$, and a natural representative of each coset is the *descending staircase* of swaps that carries that wire down. In the library a coset is not a set of circuits but a label naming one, an element of the finite type C n; here the label is the length of the staircase, as a unary numeral bounded by the width, and the *section* $[\,_\,]^{\mathsf{c}}$ maps it to the staircase circuit. The representatives on three, two, and one wires are drawn on the right:

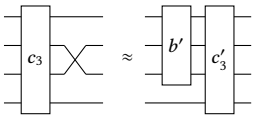
```
data C : ℕ → Set where
  ε : C n
  σ•_ : C n → C (1+ n)
[_]ᶜ : C n → Circuit (1+ n)
[ ε ]ᶜ      = ε
[ σ• c ]ᶜ = σ • ([ c ]ᶜ ↑)
```



So C n has exactly the $n{+}1$ elements drawn above for $n = 2, 1, 0$, matching the index of S_n in S_{n+1}. Why this yields a normal form is the classical argument of computational group theory [Holt et al. 2005; Sims 1994]: every $f \in S_{n+1}$ lies in one coset, say with representative c_n, so $f = f' \cdot c_n$ with $f' \in S_n$, and recursing on f' writes f uniquely as $c_1 c_2 \cdots c_n$ with $c_k \in$ C k — a numeral in the *factorial number system*, one digit per level, since $|S_{n+1}| = (n{+}1) \cdot n \cdots 2$.

Computing this factorisation is the job of the coset table. Given a coset, represented by its staircase, and one generator, the table says where their product lands — the updated coset — and what is left over on the wires above — the residual circuit — so that staircase followed by generator equals residual (shifted up) followed by updated staircase. Pushing σ into a staircase can lengthen it, cancel with it ($\sigma^2 = \varepsilon$), or pass it through and shift it up (Yang–Baxter), or commute past; for a generator that does not touch the bottom wire we can recurse. The residual is empty or a single swap here, but in richer gate sets it can be a longer circuit, which is why the table returns a circuit on one wire fewer rather than a generator (Figure 2 draws the four cases):

```
ract : C (1+ n) → Gen (2+ n) → Circuit (1+ n) × C (1+ n)
ract ε σ-gen = ε , σ• ε                  - grows
ract (σ• ε) σ-gen = ε , ε                - cancels (σ² = ε)
ract (σ• σ• c) σ-gen = σ , σ• σ• c       - passes (Yang-Baxter)
ract ε (g ↥) = [ g ]ʷ , ε                - commutes past
ract (σ• c) (g ↥) = proj₁ (ract c g) ↑ , σ• (proj₂ (ract c g))   - recurses
```



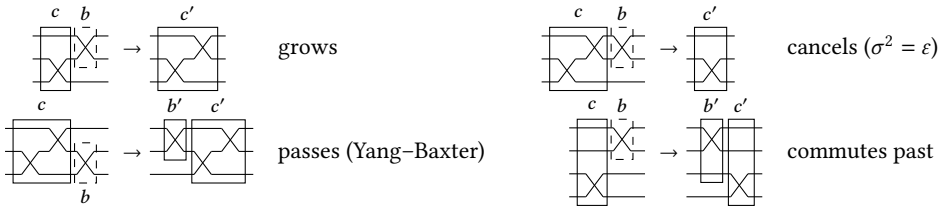


Fig. 2. Pushing a swap through a staircase: the four cases of the coset table ract as circuit rewrites.

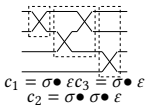
σ-gen abbreviates gate$_2$ σ-gate. The last clause is the recursion: a shifted generator meets a staircase by meeting its tail one wire down. Each clause is certified by the *soundness law*, pictured on the right and proved once per table by a short equational derivation from the axioms:

```
ract-sound : ∀ c b → let (b' , c') = ract c b in [ c ]ᶜ • [ b ]ʷ ≈ b' ↑ • [ c' ]ᶜ
```

The stateful word traversal _^{t} extends ract to whole circuits, threading the coset from left to right and concatenating the residuals; run from the identity coset, it maps an $(n{+}2)$-wire circuit to an $(n{+}1)$-wire circuit and a coset — one level of normalisation.

Iterating the level is the coset tower, whose carrier grows by one digit per level:

```
NF : ℕ → Set
NF 0 = ⊤
NF (suc k) = NF k × C k - ⊤ × C 0 × C 1 × ⋯ × C k
```



so NF n has exactly $n!$ elements. The normal-form function nf : Circuit $n \to$ NF n runs the tables; its *section* inv-nf : NF $n \to$ Circuit n rebuilds a circuit from its digits by concatenating the staircases, suitably shifted, as drawn on the right for $(\mathrm{tt}, c_0, c_1, c_2, c_3) \in \mathrm{NF}\ 4$, the digit $c_0 \in \mathrm{C}\ 0$ being the empty staircase — one staircase per level, each shifted up to the wires of its level: a "staircase of staircases".

A.4 Semantics, uniqueness, and completeness

So far everything was syntax. The development interprets circuits in the group Permutation′-group n of permutations of Fin n, with σ swapping the bottom two elements. *Soundness* — $w \approx v$ implies $[\![w]\!] \doteq [\![v]\!]$ — is a small case analysis over the axioms. The heart of the matter is the *unique normal form* property, that normal forms with equal denotations are equal: $[\![\ \mathsf{inv\text{-}nf}\ u\ ]\!] \doteq [\![\ \mathsf{inv\text{-}nf}\ v\ ]\!]$ implies $u \equiv v$. It is proved by induction on the tower: the lower digits fix the last point, so the top digit alone determines where the permutation sends it; equal denotations therefore force equal top digits, and cancelling them reduces the claim to the level below. Soundness and uniqueness are exactly the premises of the generic lemma by-normalization, which concludes *completeness*: $[\![w]\!] \doteq [\![v]\!]$ implies $w \approx v$. Together with surjectivity (every permutation is a product of adjacent transpositions), these assemble into the library's *presentation record*: a Grouplike witness, which makes the presented monoid a group, and a proof that $[\![_]\!]$ is a group isomorphism from that

group onto the target (IsGroupIsomorphism is the standard library's, between the group the witness induces and G; the record's module plumbing that says so is elided):

```
record _IsPresentationOf_ (_===_ : WRel X) (G : Group a ℓ) : Set (a ⊔ ℓ) where
  field
    gl : Grouplike _===_
    [[_]] : Word X → Group.Carrier G
    iso : IsGroupIsomorphism [[_]]
```

Every presentation theorem in the paper has the shape of this one:

```
symmetric-presentation : ∀ n → (n VRel,_===_) IsPresentationOf (Permutation′-group n)
```

read: the monoid of swap circuits modulo {order, yang-baxter} and the structural rules *is* the group of permutations of Fin n. The symplectic factor of §4 is this pipeline again, with richer gates and a richer action, and §5 composes such presentations.

B Generic Presentation Composition

The second proof interface combines presentations of smaller groups. For relations over a disjoint union of alphabets, a three-way join keeps the two factors' rules and a mixed relation:

```
data _◇_◇_ (Γ₁ : WRel A) (Γ₂ : WRel B) (Γ₃ : WRel (A ⊎ B)) : WRel (A ⊎ B) where
  left  : Γ₁ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ]ₗ [ v ]ₗ
  right : Γ₂ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) [ u ]ᵣ [ v ]ᵣ
  mid   : Γ₃ u v → (Γ₁ ◇ Γ₂ ◇ Γ₃) u v
```

Choosing the mixed relation recovers the classical constructions: free product (EmptyRel), direct product (CommRel), semidirect product (ConjRel^{w} *conj*: moving a letter h of the acting group past a letter n of the normal subgroup conjugates n, by a *word*, $h \bullet n = conj\ h\ n \bullet h$) and definitional extension (SugarRel). For each, the library lifts normal forms from the factors to the construction and proves that if the factors present concrete groups, the constructed relation set presents the constructed group. The semidirect theorem is the one §5 uses. Beyond presentations of the factors it needs two well-definedness conditions on the action: that it respects the acting group's axioms in its first argument (conj-hyph) and the normal subgroup's in its second (conj-hypn):

```
module _ (conj-hyph : ∀ {c d} n → c ===₂ d → (conj ʰ′) c n ≈₁ (conj ʰ′) d n)
         (conj-hypn : ∀ c {w v} → w ===₁ v → (conj ⁿ′) c w ≈₁ (conj ⁿ′) c v) where
  module Presentation (G1 G2 : Group 0ℓ 0ℓ)
    (p1 : Γ IsPresentationOf G1) (p2 : Δ IsPresentationOf G2) where
    dpres : (Γ ◇ Δ ◇ ConjRel^w conj) IsPresentationOf G1⋊G2
```

Here G1⋊G2 is the semidirect product of the two *semantic* groups, built from an Action record whose action is the syntactic conj transported through the two presentation isomorphisms. A further module proves the classical recipe for presenting a group *extension* $1 \to N \to G \to Q \to 1$ from presentations of N and Q, how G conjugates N, and the correction in N by which each lifted relator of Q fails to hold (cf. Proposition 2.55 of [Holt et al. 2005]): a second mixed component, RelTwist, twists each relator of the quotient by its correction, and the relation set of the recipe is

```
extension-presentation S R conj corr = S ◇ EmptyRel ◇ (ConjRel^w conj ∪ RelTwist R corr)
```

Semidirect products have trivial relator corrections. The exact qupit layer uses the central-extension recipe, but proves completeness by splitting scalar and gate words (§5.4); it does not instantiate the general theorem's bijective-normal-form premises. The direct-product theorem is then used to adjoin the sign.

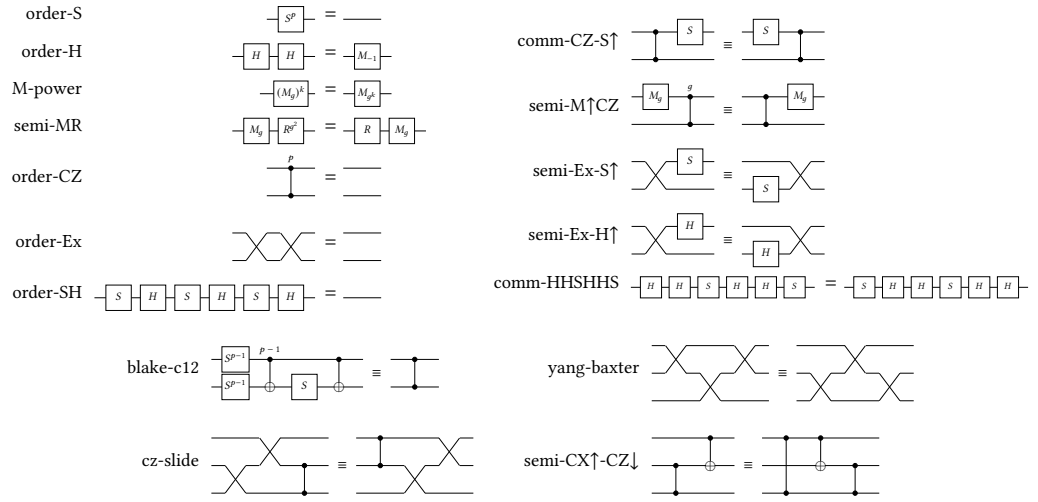


Fig. 3. The sixteen rules of `Paper-V0`'s _SRel,_===_ as circuit equations. A word is drawn left to right as written and read in matrix-multiplication order, so in $w \bullet v$ the right factor v acts first (§3.2). blake-c12 relates two circuits implementing controlled-Z; yang-baxter and cz-slide are the swap calculus.

C Projective Presentation Variants

This appendix records the relation reductions behind Table 4.

From Simplified-V1 to our sixteen rules. `Paper-V0` is our primary rule set: the qupit paper's Figure 1 read modulo scalars, with the multiplier spelled over R and with $(SH)^3 = \varepsilon$ kept as an axiom. In Agda it is one inductive family _SRel,_===_ with sixteen constructors, one per rule, each stated at the smallest number of wires at which it makes sense (two of them were shown in §3.2; the name is the same as the symplectic family's of §4.5, each rule set being its own module, and _QRel,_===_ is again its closure under the structural rules); Figure 3 shows them as circuit equations, drawn from the Agda terms, under the constructor names used throughout. The figure groups the rules by arity. As in §2, shifting equations and disjoint commutations are supplied by the circuit layer, rather than counted among these gate-specific rules. The scalar layer also identifies a scalar with its shift. Pauli commutation and the Pauli-versus-CZ rules are derived, not added as axioms.

`Paper-V0` shares its alphabet with `Simplified-V1`, so the identity on words is the candidate isomorphism. Ten of the axioms are shared verbatim; the rest is real work in both directions. `Paper-V0` axiomatises the swap directly and two-qupit interactions through blake-c12 and semi-CX↑-CZ↓, and from these its `Lemmas` derives Selinger's c10–c15 natively together with the lower-wire rules, the two Pauli-versus-CZ rules and the swap–CZ commutation; conversely the seven swap rules are proved in `Simplified-V1` by transport along the chain symplectic → simplified → semidirect → V1 rather than by fresh derivations. The module exports v1⇒pap, which transports any Simplified-V1 theorem into `Paper-V0`'s theory; it is what lets later layers reuse the calculi rather than re-prove them.

From our sixteen rules to the paper's fifteen: connecting the two routes. The paper's Figure 1 spells the multiplier over S, with an explicit Pauli prefix (its Lemma 2.14): M_x = SHS' x^{-1} x where SHS' a b = $Z^{(b-1)/2}X^{(1-a)/2}S^bHS^aHS^bH$, and correspondingly states semi-MR as $M_g \bullet S^{g^2} \bullet Z^{(g^2-g)/2} = S \bullet M_g$ (its rule C4), the Z-exponent being what is left after the $Z^{1/2}$ hidden in each R is split off and carried

across the multiplier, where it picks up a factor of g. `Paper-V1` is the paper's Figure 1 modulo scalars in this spelling: our sixteen rules with order-H, M-power, semi-MR and semi-M↑CZ restated over SHS', and *fifteen* rules rather than sixteen, because $(SH)^3 = \varepsilon$ is now a theorem. Under the S-spelling, M_1 *is* the word $(SH)^3$ — at $x = 1$ the Pauli prefix vanishes — and M-power at $k = 0$ already gives $M_1 \approx \varepsilon$; under our R-spelling M_1 is $(RH)^3$, a different word, which is why the axiom had to stay there. The isomorphism with `Paper-V0` is again the identity on words; eleven axioms cross directly, and the four multiplier rules need the bridge between the two spellings, $R^aHR^bHR^aH \approx$ SHS' $a\ b$, proved once over an abstract relation in `Shared.PauliBase` and transported by v1⇒pap. The resulting presentation is Theorem 5.1.

D External Matrix Interpretation

Write $V = \mathbb{Z}_p^{2n}$ and $\omega = e^{2\pi i/p}$. Gross's Weyl operators satisfy $W(v)W(w) = \omega^{\frac{1}{2}\mathrm{sform}(v,w)}W(v+w)$, after matching the order of the position and momentum coordinates [Gross 2006, §II.1, Eq. (4)]. Every Clifford unitary is, up to phase, a Weyl operator followed by a symplectic implementer [Gross 2006, Theorem 3]. For finite fields of odd characteristic those implementers can be chosen as a *linear* Weil representation ρ, with exact covariance $\rho(S)W(v)\rho(S)^{-1} = W(Sv)$ [Gurevich and Hadani 2007, Theorem 1.3.2]. Thus, writing $U(e, v, S) = \omega^e W(v)\rho(S)$, one obtains

$$U(e, v, S)U(f, w, T) = U\big(e + f + \tfrac{1}{2}\mathrm{sform}(v, Sw),\ v + Sw,\ ST\big).$$

This is the Heisenberg–Weil representation of the Jacobi group [Gurevich and Hadani 2010, §1.3], and its multiplication is exactly our structural extension's. For $n \geq 1$ it is faithful: conjugation on Weyl operators determines S and then v, and the remaining scalar determines e. Its scalar subgroup is therefore exactly $\langle \omega I \rangle$; at width zero the representation is simply $e \mapsto \omega^e$. Adjoining $-I$ gives a direct product with $\mathbb{Z}_2$, since p is odd, with scalar subgroup $\langle -\omega I \rangle$. These are the two finite scalar conventions used here; the full unitary normalizer allows every scalar in $U(1)$. The representation argument is mathematical background, not an Agda theorem. An isomorphism of abstract groups would not identify the named gates: one must also prove that their interpretations are the specific δ_p-normalised matrices. This gate-by-gate correspondence is not proved here. Explicit constructions on symplectic generators appear in Neuhauser [Neuhauser 2002].

E Artifact Source Inventory

Table 6 measures retained sources, including comments and blank lines, excluding the external standard library and literate excerpts. The counts are reproduced by `python3 vqupit/notes/stats.py`. Roughly 40% of the source remains in the symplectic layer and 21% in projective composition and translations. The latter cost is substantial even though there is no second large normalizer. These counts describe the proof's distribution; they do not measure saved effort, runtime, or generality.

Table 6. Retained Agda sources, measured 12 November 2026. Counts include comments and blank lines.

Component	Files	Lines
Presentation framework and supporting algebra	49	12 445
Symplectic semantics, normalization, and relations	107	30 871
Projective Pauli presentation	2	1 148
Projective Clifford composition and translations	25	15 900
Exact scalar extension	14	5 530
Original Figure 1 with scalars	22	7 329
Symmetric, cyclic, and trivial examples	26	2 773
Theorem index and notation	2	443
Total	247	76 439